The Writing Was on the Floor: A Zero-Query Fresh-Message Forgery against GLYPH

Martin Feussner
Selmer Center, University of Bergen
martin.feussner@uib.no

13 October 2026

AI disclosure and review status. This audit used the OpenAI model Daybreak Blue at the ultra reasoning level. OpenAI Codex performed AI-assisted target selection and attack-history searching, scheme reconstruction, cryptanalysis, proof development, implementation, experiment generation, and technical writing. Separate specialist AI agents performed adversarial review, and a fresh-context AI agent independently reproduced the result with a separately written implementation. These were AI review and AI reproduction, not human verification. The technical, practical, adversarial AI, independent-reproduction, final attack-history, and Publication Reviewer gates all passed before public release. No independent human verification was performed or required. OpenAI Codex generated and selected the title and intentionally chose "The Writing Was on the Floor" as wordplay on GLYPH/writing and on the zero K-floor that makes the forgery work.

Abstract

We give a public-key-only, zero-signing-query fresh-message forgery against the final revision of Arjun Chopra's *GLYPH: A New Instantiation of the GLP Digital Signature Scheme*, IACR ePrint 2017/766. The target claims 137.17 bits against its modeled forgery attack at the recommended row $(m, n, q, B, \omega) = (2048, 1024, 59393, 16383, 16)$. Its verifier hashes the K-floor of $az_1 + z_2 - tc$, where $K = B - \omega = 16367$, and checks that the result encodes to the supplied sparse challenge c.

For any attacker-chosen fresh message, we first derive c by hashing the all-zero floor polynomial. We set $z_1 = 0$ and, coefficient by coefficient, choose z_2 from $\{0, -16366, 16366\}$ so that every coefficient of $z_2 - tc$ modulo q lies in the zero-floor interval $[0, 32735)$. The response lies strictly inside the paper's K bound. The ordinary verifier therefore hashes exactly the zero polynomial used to derive c and accepts. The construction is deterministic once the public key and message are fixed, uses no secret, and makes no signing query.

The sealed primary implementation accepted 64/64 fresh-key, fresh-message forgeries under the pinned author's unmodified `glp_verify`; all 512 negative controls rejected. A separately written fresh-context AI reproduction accepted another 64/64 full-parameter forgeries and rejected all 256 controls. Each reproduction exhaustively checked the selector over all 59,393 residues. The primary attack core took 68.648–206.126 microseconds, with a mean of 81.584 microseconds. We located no previous public substantive attack against final-v3 GLYPH in searches completed through 13 October 2026. The claim concerns the numerical signature and message/hash interface specified by the paper and author source; neither defines an author-standardized portable signature wire format.

1 Introduction

GLYPH is a 2017–2018 re-instantiation of the GLP lattice signature scheme [1, 2]. It retains GLP's Ring-LWE-style key relation and Fiat–Shamir structure, changes the recommended parameters, and introduces Compress2 and a new K-floor operation. The target presents its recommended row as providing 137 bits of conservative security and estimates the cost of its modeled forgery attack as $2^{137.17}$ basic classical operations [1].

The verifier's floor operation creates a much simpler route. A forger need not recover a secret or solve the short-vector problem modeled by the target. It can decide in advance which floor polynomial the verifier will hash, derive the challenge from that public value and a chosen message, and then place each

response coefficient into the corresponding floor bucket. For the recommended modulus and bounds, the zero bucket is reachable for every possible coefficient of $-tc$ using one of three strictly bounded values.

The resulting attack has the following properties:

- it targets the final ePrint object and the complete recommended parameter row, without weakening any parameter;
- it works for every public key and any attacker-chosen message;
- it produces the numerical tuple (z_1, z_2, c) accepted by the ordinary verifier and uses the target's normal message-hash path;
- it requests no signature and uses no private value; and
- it is a design-level failure of the specified K-floor verification relation, rather than a parser, randomness, fault, or compatibility defect.

Technique attribution. We make no novelty claim for constructing a transcript backward from a public verification equation, for choosing a public hash target before solving for a response, or for manipulating a rounding bucket. The target itself analyzes a generic forgery strategy that chooses a hash target and then seeks a short preimage with BKZ [1]. Our contribution is the observation and full-parameter demonstration that GLYPH's specific K-floor relation makes such a preimage available coefficient by coefficient without lattice reduction.

There is also earlier, distinct compression-related work in the GLP lineage. Ducas, Durmus, Lepoint, and Lyubashevsky observed that the original GLP12 compression workaround permits changing a least-significant bit in an already valid signature while preserving the checked high bits with high probability [3]. They explicitly identify this as a failure of strong unforgeability. That method starts with an existing signature and produces another signature on the same message; it predates GLYPH's Compress2/K-floor construction. It is therefore not the zero-query fresh-message attack proved here.

2 Exact target, history, and claim boundary

2.1 Pinned final revision

The target is *GLYPH: A New Instantiation of the GLP Digital Signature Scheme* by Arjun Chopra, IACR ePrint 2017/766 [1]. The audit uses the current 14-page final object, operationally called final v3, last revised 7 June 2018. Its exact PDF has SHA-256 digest

`ff0eeaea3a9fcb830197ea954472869a08adbf42e64e727470fa8e0038c6682a.`

The extracted text has SHA-256 digest

`8ecd84282a5a752e1c2256597b9a10bdf50bfdb8eb2c572fa34eba27e54c1f2a.`

The paper links the author's public-domain implementation. We pin that repository at commit

`74b96d691391f687fe8f91e1d5cb3c251762d7cc`

with Git tree

`2ae8eec4b23dc57fc816c8935da8c2a59a753196.`

The retained source archive has SHA-256 digest

`43c68dfab7e717bf37ba57be9aa2fc49270f8eb42bfa420c85bf473aca12fe60.`

The pinned source uses the same parameters and implements the paper's numerical verification equation.

2.2 Claimed security and recommended parameters

Table 1 gives the sole recommended GLYPH row. The paper instantiates the hash with SHA-256, maps its output to a challenge of Hamming weight $\omega = 16$, and reports a 1.8 kB signature, 0.3 kB secret key, and 2.0 kB public key. It estimates primal key recovery, dual key recovery, and forgery at $2^{139.5}$, $2^{175.8}$, and $2^{137.17}$ basic classical operations, respectively. These are the target's estimates, not claims revalidated by this audit. The attack below bypasses each modeled lattice problem.

Table 1: The audited full parameter row.

m	n	q	B	ω	$K = B - \omega$	$D = 2K + 1$	claimed forgery
2048	1024	59393	16383	16	16367	32735	$2^{137.17}$

2.3 Dated attack-history result

We located no previous public substantive attack against final-v3 GLYPH in our searches completed through 13 October 2026. The final search repeated exact-title, ePrint-identifier, author, repository, K-floor, zero-query, fresh-message, forgery, attack, and cryptanalysis queries; checked the official record and author repository; screened the five citing works reported by OpenAlex; and revisited the GLP lineage. This is a bounded, dated negative search result, not an absolute priority claim.

The closest located result is the original-GLP same-message malleability just described [3]. Located induced-fault work attacks original GLP under faults rather than final-v3 GLYPH under ordinary execution [4]. Masking work that used the GLYPH code base expressly focused on countermeasures and did not report this numerical-verifier attack [5].

2.4 Intended interface

The target paper defines a signature as the numerical tuple (z_1, z_2, c) and defines verification directly over that tuple and a message. The author source exposes the same tuple as an in-memory structure, accepts message bytes, and implements the same hash and floor equation. This is sufficient to establish a design-level attack on the declared numerical signature interface.

Neither the final paper nor the author source specifies a portable external signature byte format or parser. In particular, the source's `poly2bytes` operation serializes a polynomial for hashing; it is not an external signature codec. We therefore make no claim about an unspecified document wrapper, protocol envelope, or deployment-specific wire encoding. The experimental codecs transport the numerical fields without changing their semantics and are not presented as author-standardized formats.

3 The GLYPH verifier

3.1 Ring and floor notation

For the recommended power-of-two conductor, let

$$\mathcal{R} = \mathbb{Z}[x]/(x^{1024} + 1), \qquad \mathcal{R}_q = \mathcal{R}/q\mathcal{R}, \qquad q = 59393.$$

Coefficients of elements in $\mathcal{R}_q$ are represented modulo q. A centered coefficient has its representative in $[-(q-1)/2, (q-1)/2]$.

Set $K = B - \omega = 16367$ and $D = 2K + 1 = 32735$. For an integer x, write the canonical residue $x \bmod q$ uniquely as

$$x \bmod q = rD + s, \qquad 0 \le s \le 2K.$$

GLYPH defines $\lfloor x \rfloor_K = r$ and applies this operation coefficient by coefficient to ring elements. Thus every residue in $[0, D)$ has K-floor zero.

3.2 Keys, signing, and verification

The public domain parameter is $a \in \mathcal{R}_q$. Key generation samples ternary $s, e \in \mathcal{R}_q$ and publishes

$$t = as + e,$$

while (s, e) remains secret. To sign a message μ, the signer samples bounded y_1, y_2, computes

$$c = \mathsf{F}(\mathsf{H}(\lfloor ay_1 + y_2 \rfloor_K \,\|\, \mu)),$$

forms $z_1 = sc + y_1$ and $z_2 = ec + y_2$, rejects samples outside the coefficient bound, and applies Compress2 to z_2. The challenge encoder F returns a ring element with exactly $\omega = 16$ coefficients in $\{+1, -1\}$ and all remaining coefficients zero.

Given (μ, z_1, z_2, c), the verifier first checks the coefficient bounds for z_1 and z_2. It then computes

$$h = az_1 + z_2 - tc \pmod{q}, \tag{1}$$

and accepts exactly when

$$c = \mathsf{F}(\mathsf{H}(\lfloor h \rceil_K \,\|\, \mu)). \tag{2}$$

The author's source follows this path in `glp_verify`, using two little-endian bytes per floor coefficient as the internal hash encoding, then hashing the message bytes and applying its sparse encoder.

Correctly generated signatures satisfy

$$az_1 + z_2 - tc = (as + e)c + ay_1 + y_2 - (as + e)c = ay_1 + y_2$$

before compression, and Compress2 is intended to preserve the K-floor. Verification therefore reconstructs the same hash input used by the signer.

4 Zero-query fresh-message forgery

4.1 Attack algorithm

Define

$$L = K - 1 = 16366, \qquad D = 2K + 1 = 32735, \qquad q - L = 43027.$$

The public-key-only forger takes (a, t) and an attacker-chosen message μ as input:

1. Let $w = 0 \in \mathcal{R}_q$ and compute
$$c = \mathsf{F}(\mathsf{H}(w \,\|\, \mu))$$
using GLYPH's prescribed hash serialization and sparse encoding.
2. Set $z_1 = 0 \in \mathcal{R}_q$.
3. Compute the public ring product $-tc \in \mathcal{R}_q$. For each coefficient, let $u \in [0, q)$ be its canonical representative and set
$$z_2(u) = \begin{cases} 0, & 0 \le u < D, \\ -L, & D \le u < q - L, \\ +L, & q - L \le u < q. \end{cases} \tag{3}$$
4. Output (z_1, z_2, c) as the signature of μ.

The algorithm never invokes the signer. Its only nontrivial algebra is multiplication of the public polynomial t by the public sparse challenge c.

4.2 Strict-interior selector proof

Lemma 1. *For every residue $u \in [0, q)$, the value $z_2(u)$ from Equation (3) satisfies $|z_2(u)| < K$ and*

$$(u + z_2(u)) \bmod q \in [0, D).$$

Proof. The only possible centered response values are 0 and $\pm L$, and $L = K - 1$, so the strict coefficient bound holds. The three selector branches give the following reconstructed intervals:

$$\begin{aligned} 0 \le u < D &\implies u + 0 \in [0, 32734], \\ D \le u < q - L &\implies u - L \in [16369, 26660], \\ q - L \le u < q &\implies u + L - q \in [0, 16365]. \end{aligned}$$

Each interval lies inside $[0, D) = [0, 32735)$. □

Theorem 1. *For every GLYPH public key at the recommended row and every message μ, the public algorithm above produces a signature accepted by the ordinary GLYPH verifier. It makes zero signing queries and uses no secret input.*

Proof. The response values meet even the paper's early strict K-bounded wording, and $z_1 = 0$ is also strictly bounded. The verifier's reconstructed polynomial is

$$h = az_1 + z_2 - tc = z_2 - tc.$$

For each coefficient, $u = (-tc)_i \bmod q$. The lemma shows that $(u + (z_2)_i) \bmod q$ belongs to $[0, D)$, so its K-floor is zero. Hence

$$\lfloor h \rfloor_K = 0 = w.$$

The verifier's challenge recomputation in Equation (2) is therefore

$$\mathsf{F}(\mathsf{H}(0 \,\|\, \mu)) = c,$$

which is exactly how the attacker chose c. The verifier accepts. □

No Fiat–Shamir circularity. The attacker does not search for a hash fixed point and does not program the hash. It chooses the verifier's intended floor output $w = 0$ first, hashes that public value with μ to obtain c, and only then solves the public coefficient relation for z_2. The selector succeeds for every possible coefficient of the resulting $-tc$.

Freshness and success probability. The attacker chooses μ and never submits it, or any other message, to a signing oracle. It therefore produces a fresh-message signature with zero queries. The proof covers every public key, every challenge output, and every message. In the mathematical interface the success probability is one; no favorable-key, favorable-message, or random-residue event is needed.

4.3 Complexity

The challenge c has $\omega = 16$ nonzero coefficients. Computing tc as a sum of signed negacyclic shifts takes $O(n\omega)$ modular operations; the selector and serialization take $O(n)$ work. Hashing and sparse encoding add their ordinary fixed-parameter costs. Memory use is $O(n)$. At $n = 1024$ this is a small public computation, in contrast with the target's modeled block-469 BKZ forgery.

5 Full-parameter experimental evidence

Both reproductions use the complete recommended row and fresh OS-random keys. Together they cover 64 primary plus 64 independently reproduced forgeries. The measured experiments support the implementation claim; the all-residue proof above establishes generality beyond the sampled keys.

5.1 Primary reproduction

The primary harness compiles the pinned author source without cryptographic changes. OpenSSL 3 no longer exports the low-level `AES_ctr128_encrypt` entry point used by the 2018 source, so the harness supplies a narrow API adapter through OpenSSL's `CRYPTO_ctr128_encrypt` and `AES_encrypt`. Before any attack trial, the upstream deterministic vector passes, exercising key derivation, signing, verification, hashing, sparse encoding, compression, and the adapter.

Key generation, forging, and verification run as separate processes. The isolated forger accepts only a canonical public key and a message. Its binary links neither the GLYPH key-generation/signing object nor any signing symbol. The separately linked verifier contains the pinned `glp_verify` symbol and no signing symbol. No secret is serialized or passed to the forger, and the recorded signing-query count is zero.

For each of 64 trials, the driver creates a fresh OS-random key and a fresh attacker message, invokes the public forger, and submits the tuple to the ordinary verifier. All 64 forgeries accept. All $64 \cdot 1024 = 65{,}536$ reconstructed coefficients have zero floor. Public-key, message, signature, and challenge fingerprints are distinct in all 64 trials.

The primary attack-core time ranged from 68,648 to 206,126 nanoseconds; the median was 76,994 nanoseconds and the mean was 81,584 nanoseconds. The complete 64-trial driver took 1.16 seconds wall-clock and used 53,760 KiB peak resident memory. It used one worker and exited successfully.

Table 2: Sealed full-parameter results.

Measurement	Primary	Fresh-context reproduction
fresh OS-random keys/messages	64	64
ordinary-verifier accepts	64/64	64/64
zero-floor coefficients	65,536/65,536	65,536/65,536
negative-control rejects	512/512	256/256
selector residues checked	59,393	59,393
selector failures	0	0
signing queries	0	0
workers	1	1

5.2 Primary controls

Each trial applies eight negative controls. Changed, truncated, and extended messages all reject. Flipping a challenge sign and changing a challenge position reject. A response change that remains within the coefficient bound but forces a nonzero floor rejects. Finally, out-of-range z_1 and z_2 coefficients reject at the verifier's bound check. These controls exercise the hash binding, sparse-challenge comparison, floor relation, and response bounds; they rule out an accidentally permissive harness.

The driver also enumerates every $u \in [0, 59393)$ for the strict-interior selector in Equation (3). It separately checks the author's inclusive COMPRESS2 boundary selector. Both exhaustive checks pass all 59,393 residues with zero failure. The attack uses only the strict-interior selector.

5.3 Fresh-context independent AI reproduction

A fresh-context AI agent received the public target pins, public attack description, and reproduction requirements. It did not read the primary package. It independently implemented challenge derivation, negacyclic sparse multiplication, and coefficient selection in C, then used the pinned author implementation only for legitimate setup and the ordinary `glp_verify` call.

For 64 fresh OS-random keys and 48-byte fresh messages, all 64 independent forgeries accepted. Changed-message, changed-challenge, in-bound changed-floor, and out-of-bound controls rejected in every trial, for 256/256 rejections. Its independently written selector also passed all 59,393 residues, and all 65,536 reconstructed trial coefficients had zero floor. The experiment body took 2.596779495 seconds; external timing was 3.50 seconds with 5,120 KiB peak resident memory and one worker.

The combined independent setup/reproduction binary necessarily links the complete upstream object, so unused signing symbols are present. Source and disassembly review found no call to `glp_sign` or `glp_deterministic_sign`. The independent forging function accepts only a public key and message, and the harness cleanses the setup secret before calling it. This is fresh-context AI reproduction, not independent human verification.

5.4 Evidence integrity

The primary machine-readable results, package manifest, and checksum list have SHA-256 digests

Artifact	SHA-256
primary `results.json`	`3d48ec90fdab886323e189359f45e46851113eb9d74a36b76dccc5219a2acc52`
primary `manifest.json`	`37f35f099485fd61cdd001a3ac76425d06c141bf5e3b9a87951ade100cabf329`
primary `MANIFEST.sha256`	`595e948beecd42200e38a36b0d6c370b474c55fa5de90ca593b53d023816f53d`
independent `MANIFEST.sha256`	`f2ce4a22e576f662bfa1b761c806916c60d120bd6dd58fddd5691f716621c6a7`
independent source	`4fb82963255a7e4ac50c742c29591163e6a899481adf6964d84b91218a60b37b`

All 37 entries in the primary checksum list and all 52 entries in the independent list verify. The primary isolated-forger source has SHA-256 digest

`26d69afea2af2ea7f82fe56dbf77460419eb906dabc1b965928bb53f96fde095.`

6 Why the verification equation fails

GLYPH's modeled forgery in Section 5.3 of the target begins with a uniform ring element y, hashes (y, μ) to obtain c, and asks BKZ to find bounded z_1, z_2 satisfying $az_1 + z_2 = tc + y$. The security estimate treats matching the complete ring element y as a short-preimage problem.

The actual verifier does not hash $az_1 + z_2 - tc$ itself. It hashes only its K-floor. Every one of the 59,393 possible residues can be shifted into the zero-floor bucket by a response in $\{0, -(K-1), K-1\}$. Setting $z_1 = 0$ therefore decomposes the apparent ring preimage problem into 1,024 independent three-choice operations. Challenge sparsity makes the remaining public product especially cheap, but the proof does not rely on favorable challenge positions or signs.

The weakness is in the mathematical verification relation. The OpenSSL adapter only restores an API removed since publication and is covered by the upstream vector. The attack uses valid numerical fields and ordinary hash and challenge derivation; it does not exploit malformed input, a parser, nonce reuse, weak randomness, a fault, timing leakage, or undefined behavior.

7 Repair discussion and limitations

Preventing $z_1 = 0$ alone does not repair the underlying issue: the security argument must account for the quotient information revealed by K-floor and show that an attacker cannot choose a floor target first and solve for a bounded response afterward. Likewise, banning the three response values used here would address one instantiation of the selector rather than the public reachability problem.

A repair must redesign the transcript binding or compression relation so that producing a bounded response for a challenge derived from an attacker-selected verifier image is computationally hard. Any proposed change should be accompanied by a proof for the exact compressed verifier, including modular wraparound and every accepted response value, and by a fresh parameter analysis. Adding an independently specified outer document hash does not change this attack: the construction works for whatever message bytes reach the declared GLYPH hash interface.

Our result is limited in the following ways:

- it attacks the final-v3 numerical tuple and message/hash interface specified by the pinned paper and source;
- it does not claim an author-standardized external signature wire format, because none is defined by those artifacts;
- it does not claim secret-key recovery, signing-key recovery, or an attack on every GLP-family design; and
- it makes no novelty claim for verification-first construction, public rounding-bucket manipulation, or the earlier GLP12 same-message malleability method.

The exact result is stronger than a same-message alternate-signature attack: the adversary selects a fresh message and needs no prior signature. It is also independent of the paper's ambiguity between early strict K-bounded wording and the later inclusive verifier boundary, because all constructed response coefficients have magnitude at most $K - 1$.

8 Reproducibility

The public package provides two entry points. From the attack directory, `python3 reproduce.py` performs the quick eight-trial full-parameter run, while `python3 reproduce-all-rows.py` performs the complete 64-trial run. Each entry point rebuilds the pinned upstream verifier and the public C forger, exhaustively checks all 59,393 selector residues, creates fresh keys and messages, submits every forgery to the ordinary verifier, runs all controls, records resources and per-trial public digests, and verifies its checksum manifest. The underlying commands are `./reproducer/reproduce.sh -quick` and `./reproducer/reproduce.sh -full`.

Reproduction requires a C compiler, Python 3 for the public wrapper, OpenSSL development headers and library, POSIX shell utilities, and GNU `time` for the reported external resource record. Both modes use one worker. Random keys and messages intentionally change across runs, so the expected invariant is the reported acceptance, rejection, bound, floor, and exhaustive-residue counts rather than byte-identical trial fingerprints. The target PDF, source commit and tree, source-file hashes, commands, environment,

binaries, symbol reports, results, and manifests are retained by each run so that the exact evidence boundary can be checked independently.

9 Conclusion

At GLYPH's complete recommended parameter row, every coefficient of the public product $-tc$ can be moved into the verifier's zero K-floor bucket with one of three strictly bounded response values. A forger can consequently choose any fresh message, derive the public sparse challenge from an all-zero floor target, set $z_1 = 0$, and solve for z_2 coefficient by coefficient. The ordinary verifier accepts for every key and message, with zero signing queries.

The sealed primary and fresh-context AI reproductions accepted 64+64 full-parameter forgeries, rejected 512+256 controls, and independently exhausted all 59,393 selector residues. The primary attack core averaged 81.584 microseconds, bypassing the target's claimed $2^{137.17}$-operation forgery cost. The result is a practical design-level fresh-message forgery against final-v3 GLYPH's declared numerical signature interface.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

[1] A. Chopra. GLYPH: A New Instantiation of the GLP Digital Signature Scheme. IACR Cryptology ePrint Archive, Report 2017/766, final revision, 2018. https://eprint.iacr.org/2017/766.

[2] T. Güneysu, V. Lyubashevsky, and T. Pöppelmann. Practical Lattice-Based Cryptography: A Signature Scheme for Embedded Systems. In *Cryptographic Hardware and Embedded Systems—CHES 2012*, LNCS 7428, pp. 530–547. Springer, 2012. https://doi.org/10.1007/978-3-642-33027-8_31.

[3] L. Ducas, A. Durmus, T. Lepoint, and V. Lyubashevsky. Lattice Signatures and Bimodal Gaussians. In *CRYPTO 2013, Part I*, LNCS 8042, pp. 40–56. Springer, 2013; IACR ePrint Report 2013/383. https://eprint.iacr.org/2013/383.

[4] N. Bindel, J. Buchmann, and J. Krämer. Lattice-Based Signature Schemes and Their Sensitivity to Fault Attacks. IACR Cryptology ePrint Archive, Report 2016/415, 2016. https://eprint.iacr.org/2016/415.

[5] G. Barthe, S. Belaïd, T. Espitau, P.-A. Fouque, B. Grégoire, M. Rossi, and M. Tibouchi. Masking the GLP Lattice-Based Signature Scheme at Any Order. In *EUROCRYPT 2018, Part II*, LNCS 10821, pp. 354–384. Springer, 2018; IACR ePrint Report 2018/381. https://eprint.iacr.org/2018/381.